swift hire

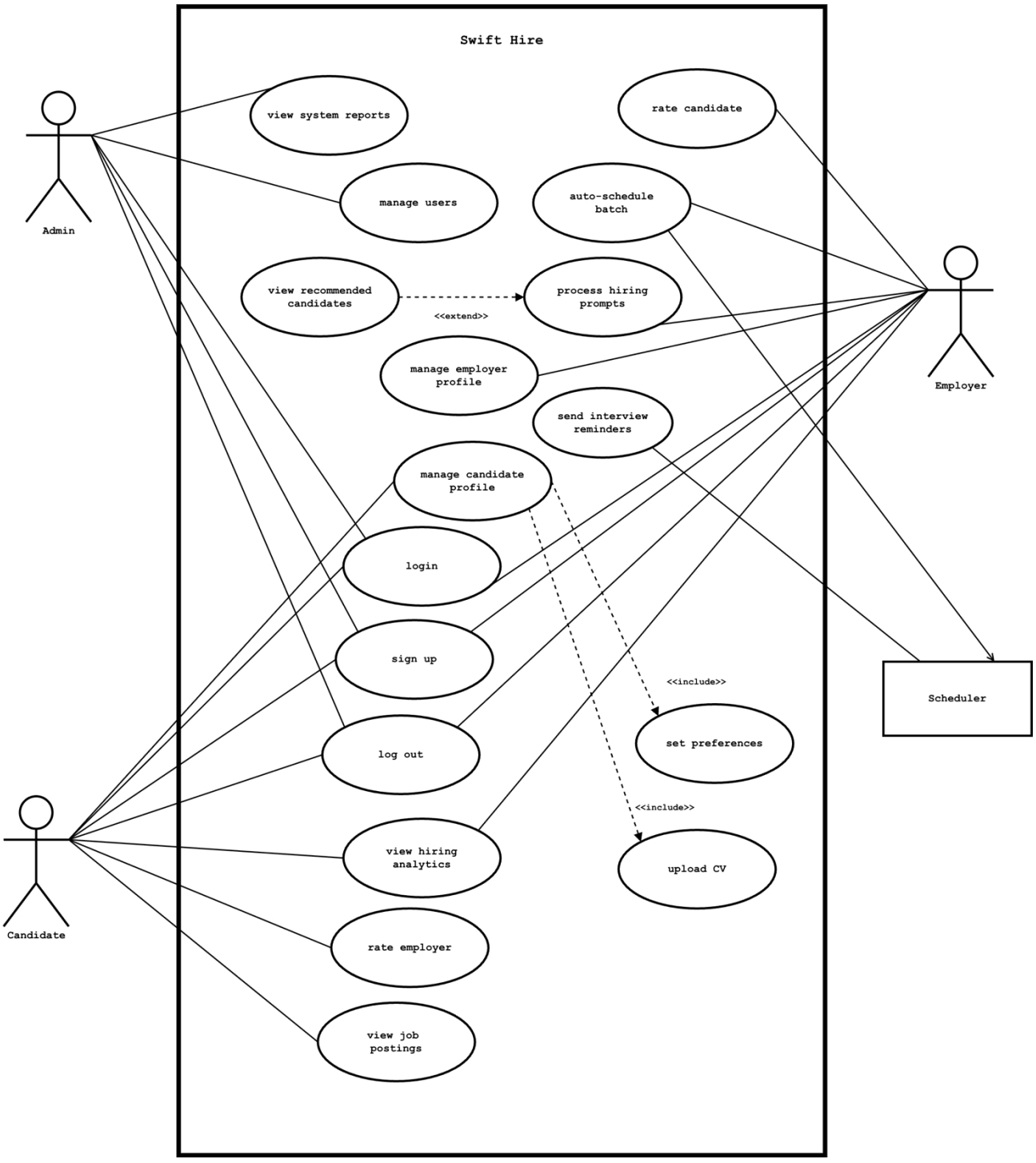
Phase 1

Team 1

Member Name	Member Roll #	Primary Responsibility
Saad Mehmood	24L-3050	UCD, ACD, NFR, UC (1-17)
Natiq Ali	23L-0882	UCD, UC-15, UC-16, NFR
Abdul Muiz	23L-3097	UCD, UC-6, UC-7, UC-9, UC-10, UC-11, UC-12
Suleman Ahmed	24L-3072	UCD, ACD, UC-8, UC-17
Ammar Arif	24L-3067	UCD, UC-13, UC-14, NFR
M. Waleed	24L-3035	UCD, UC-1, UC-2, UC-3, UC-4, UC-5

Table of Contents

1. UCD


Swift Hire
Admin
view system reports
manage users
view recommended candidates
<<extend>>
process hiring prompts
manage employer profile
send interview reminders
manage candidate profile
login
sign up
log out
view hiring analytics
rate employer
view job postings
rate candidate
auto-schedule batch
Employer
<<include>>
set preferences
<<include>>
upload CV
Scheduler
Candidate

2. Use Cases

2.1 Login

Identifier	UC-01
Name	Login
Summary	A user enters email and password to login. The system validates the credentials and grants access if correct. If not, the system shows an error and allows retry.
Priority	HIGH
Actor(s)	Candidate, Employer, Admin
Pre-condition(s)	The user already has an account (account exists in the system database) and the user is currently logged out (no active authenticated session).
Post-condition(s)	User is redirected to the correct dashboard based on role (Candidate / Employer / Admin).

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	User navigates to the login page	
2		Displays the login page
3	User enters email/username and password	
4		Validates input format / shows "required field" warnings
5	User clicks the Login button	
6		Verifies credential against the database
7		Creates session and redirects to the correct dashboard

Alternate Course of Action (Empty Input Fields)		
S#	**Actor Action**	**System Response**
5	User clicks "Login" without entering required fields.	

6		Displays message: "Email and password are required."
7	User enters the missing details.	
Process resumes at Step 5 of the typical flow.		
Alternate Course of Action (Invalid Input Format)		
S#	**Actor Action**	**System Response**
5	User violates the standard email format and presses "Login" button.	
6		Displays message: "Enter a valid email address."
7	User enters a valid email address.	
Process resumes at Step 5 of the typical flow.		
Alternate Course of Action (Wrong Credentials)		
S#	**Actor Action**	**System Response**
5	User enters incorrect email or password and presses "Login" button.	
6		Displays message: "Invalid email or password."
7	User re-enters correct credentials.	
Process resumes at Step 5 of the typical flow.		
Alternate Course of Action (Account Not Found)		
5	User enters unregistered email and presses "Login" button.	
6		Displays message: "Unregistered email, please register first."
7	User re-enters a correct email address.	
Process resumes at Step 5 of the typical flow.		

2.2 Process Hiring Prompt

Identifier	UC-02
Name	Process Hiring Prompt
Summary	Employer enters a hiring prompt in one text box. The system validates the prompt, extracts requirements using Regex and dictionary tables (skills/experience/location/shift), saves a job request, and generates a ranked candidate match list for viewing.
Priority	HIGH
Actor(s)	Employer
Pre-condition(s)	Employer is logged in. Candidate profiles and Known Skills/Locations/Shifts dictionary data exist in the database.
Post-condition(s)	Parsed hiring prompt is saved as a job request/posting, and candidate match results are generated (ready to be shown in UC-03).

Typical Course of Action

S#	Actor Action	System Response
1	Employer opens the hiring prompt screen	
2		System displays the prompt input field and the "Submit" button
3	Employer types the hiring prompt (e.g., required skills and experience)	
4	Employer clicks the "Submit" button	
5		System verifies the input prompt.
6		System parses the prompt using Regex and matches terms with the dictionary tables.
7		System saves extracted requirements as a job request/posting in the database.
8		System runs ATS scoring and prepares a ranked list of recommended candidates.

Alternate Course of Action (Empty Prompt)		
S#	**Actor Action**	**System Response**
4	Employer clicks "Submit" without entering required details.	
5		Displays message: "Prompt cannot be empty."
6	Employer enters the missing details.	
Process resumes at Step 4 of the typical flow.		
Alternate Course of Action (No Recognizable Skills/Requirements/Unclear Values)		
S#	**Actor Action**	**System Response**
6		System detects that employer's prompt has unclear skills/values/requirements.
7		Displays message: "Could not identify skills/requirements. Please rewrite the prompt."
8	Employer rewrites the prompt with clarity.	
Process resumes at Step 4 of the typical flow.		
Alternate Course of Action (Insufficient Database Records)		
S#	**Actor Action**	**System Response**
6		System detects that database doesn't have sufficient matching data.
7		System shows: "No matching candidates found. Try changing skills/experience/location."
8	Employer rewrites the prompt with changed details.	
Process resumes at Step 4 of the typical flow.		

2.3 View Recommended Candidates

Identifier	UC-03
Name	View Recommended Candidates
Summary	After a recruiter processes a hiring prompt, the system shows a ranked list of matching candidates with match scores. The recruiter can review profiles and select preferred candidates for interview scheduling. Extends [UC-02: Process Hiring Prompt].
Priority	HIGH
Actor(s)	Employer
Pre-condition(s)	Employer is logged in. UC-02 (Process Hiring Prompt) has been completed successfully, and a job request exists. Matching results are available in the system.
Post-condition(s)	Recommended candidates list is displayed. Selected candidates (if any) are saved for UC-04 (Auto-Schedule Batch).

Typical Course of Action

S#	Actor Action	System Response
1	Employer selects 'View Recommended Candidates' for the processed prompt.	
2		System retrieves matched candidates and displays a ranked list with match scores.
3	Employer clicks a candidate to view details.	
4		System displays the selected candidate’s profile (skills, experience, summary).
5	Employer selects preferred candidates and clicks 'Proceed'.	
6		System validates selection count, saves selected candidates for scheduling, and confirms.

7	Employer clicks 'Continue to Scheduling'.	
8		System navigates to UC-04 (Auto-Schedule Batch) / interview timeline screen.
Alternate Course of Action (No Candidates Selected)		
S#	**Actor Action**	**System Response**
5	Employer clicks 'Proceed' without selecting any candidates.	
6		System displays: "You must select at least 1 candidate."
7	Employer selects at least 1 candidate.	
Process resumes at Step 5 of the typical flow.		
Alternate Course of Action (Candidate Profile Unavailable)		
S#	**Actor Action**	**System Response**
4		System faces a problem in fetching the candidate profile data and displays: "Candidate profile cannot be opened right now. Try again later."
Process resumes at Step 2 of the typical flow.		

2.4 Auto-Schedule Batch

Identifier	UC-04
Name	Auto-Schedule Batch
Summary	Recruiter provides availability windows and the system automatically schedules 45-minute interviews (no overlap, no overflow), generates meeting links, and sends invitations.
Priority	HIGH
Actor(s)	Employer
Pre-condition(s)	Employer is logged in. UC-03 is completed and the recruiter has selected candidates for interview scheduling.

Post-condition(s)	Interview schedule is saved. Meeting links are generated and sent to selected candidates. Confirmation is shown to the recruiter.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Employer opens Auto Schedule Batch for the selected candidates.	
2		System shows the scheduling screen and asks for availability windows.
3	Employer enters availability window (date, start time, end time).	
4	Employer clicks 'Generate Schedule'.	
5		System validates the window is available and checks that 45-minute slots can fit.
6		System assigns candidates into non overlapping 45-minute slots within the window and shows a schedule summary.
7	Employer reviews the schedule summary and clicks Confirm.	
8		System creates meeting links (Calendly) and sends invitations to the candidates.
9	Employer clicks OK/Finish.	
10		System stores the final schedule and displays a confirmation message.

Alternate Course of Action (Insufficient Time Window)		
S#	**Actor Action**	**System Response**
4	Employer enters insufficient availability window and clicks 'Generate Schedule'.	
5		System displays: “Not enough time for 45-minute slots. Add a longer window.”

6	Employer updates the availability window to meet the requirements.	
Process resumes at Step 4 of the typical flow.		
Alternate Course of Action (Invalid Time Window)		
S#	**Actor Action**	**System Response**
4	Employer enters invalid availability window and clicks 'Generate Schedule'.	
5		System displays: "Invalid time range. End time must be after start time."
6	Employer enters valid availability window to meet the requirements.	
Process resumes at Step 4 of the typical flow.		
Alternate Course of Action (Calendly/Email Service Failure)		
S#	**Actor Action**	**System Response**
8		System displays: "Schedule created, but invitations could not be sent. Try again later."
Process resumes at Step 7 of the typical flow.		
Alternate Course of Action (Already Scheduled Time (Conflict))		
S#	**Actor Action**	**System Response**
4	Employer enters overlapping availability window and clicks 'Generate Schedule'.	
5		System detects a scheduling conflict and displays: "Selected time window is not available. Please choose a different time window."
6	Employer enters disjoint availability window to meet the requirements.	
Process resumes at Step 4 of the typical flow.		

2.5 Rate Candidate

Identifier	UC-05
Name	Rate Candidate
Summary	After an interview (or no-show), the recruiter rates the candidate (1–5) and optionally leaves feedback. The system saves the rating and updates the candidate's overall rating for future hiring decisions.
Priority	HIGH
Actor(s)	Employer
Pre-condition(s)	Employer is logged in. A candidate exists and has a completed interview record (or marked no-show) for the employer.
Post-condition(s)	Candidate rating is stored with optional feedback. Candidate overall rating is updated and available for future searches.

Typical Course of Action

S#	Actor Action	System Response
1	Employer opens a candidate's interview entry and selects 'Rate Candidate'.	
2		System displays the rating form (1–5) and an optional feedback/comment field.
3	Employer selects a rating (1-5 stars shown) and (optional) enters feedback.	
4	Employer clicks 'Submit Rating'.	
5		System validates that some rating is given and feedback, if provided, is within limits.
6		System saves the rating, updates the candidate's overall rating, and stores the feedback.
7		System shows a confirmation message: "Rating submitted successfully."

8		Employer returns to the interview list / dashboard.
Alternate Course of Action (Rating Is Missing)		
S#	**Actor Action**	**System Response**
4	Employer clicks 'Submit Rating' without selecting a rating.	
5		System displays: "Please select a valid rating (1–5)."
6	Employer chooses a valid rating.	
Process resumes at Step 4 of the typical flow.		
Alternate Course of Action (Already Rated for This Interview)		
S#	**Actor Action**	**System Response**
4		System detects the candidate has already been rated for this interview and displays: "You have already rated this candidate for this interview."
Process resumes at Step 8 of the typical flow.		

2.6 Sign Up

Identifier	UC-06
Name	Sign Up (Create Account)
Summary	This use case describes the process of a new user (Candidate or Employer) creating an account in Swift Hire.
Priority	High
Actor(s)	Candidate, Employer
Pre-condition(s)	The user must have a valid email address. The user must not already have an existing account with the same email. The system database must be operational.

Post-condition(s)	A new user account is successfully created and stored in the system database. The user is authenticated and redirected to their respective dashboard.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	User opens the Swift Hire web application.	
2		System displays the homepage with "Sign Up" option.
3	User clicks on "Sign Up".	
4		System displays the registration form.
5	User enters required details (name, email, password, role selection: Candidate or Recruiter).	
6	User clicks "Register".	
7		System validates the entered information.
8		System creates the new user account and stores information in the database.
9		System prompts for "Successful Registration" and redirects to homepage.
Alternate Course of Action (Email Already Exists)		
S#	**Actor Action**	**System Response**
6	User enters an email that already exists.	
7		System displays error message: "Email already registered. Retry."
8	User enters a new email.	
Process resumes at Step 6 of the typical flow.		
Alternate Course of Action (Invalid Email Format)		
S#	**Actor Action**	**System Response**

5	User enters invalid email format.	
6		System underlines the email and displays: "Invalid email format".
7	User enters a valid email.	
Process resumes at Step 6 of the typical flow.		
Alternate Course of Action (Required Field Missing)		
S#	**Actor Action**	**System Response**
6	User clicks "Register" without filling-in the required fields.	
7		System displays: "Fill all the required fields".
8	User enters the missing details.	
Process resumes at Step 6 of the typical flow.		
Alternate Course of Action (Weak Password)		
S#	**Actor Action**	**System Response**
5	User enters a weak password (e.g. 1122).	
6		System displays: "Please enter a strong password. Minimum 8 characters".
7	User updates the password to meet the requirements.	
Process resumes at Step 6 of the typical flow.		

2.7 Log Out

Identifier	UC-07
Name	Log Out
Summary	This use case describes the process of a logged-in user (Candidate, Admin or Employer) logging out of the Swift Hire.
Priority	High
Actor(s)	Candidate, Employer, Admin

Pre-condition(s)	The user must be logged into their Swift Hire account and have an active session. The system must be operational and able to manage session termination.
Post-condition(s)	The user session is terminated successfully, and the user is redirected to the homepage or login page. The system ensures that unauthorized access to the account is prevented after logout.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	User is on the dashboard or any page.	
2		System displays the "Log Out" option.
3	User clicks on "Log Out".	
4		System terminates the active session.
5		System clears session data and authentication tokens.
6		System redirects the user to the homepage or login page.
Alternate Course of Action (Session Expired)		
S#	**Actor Action**	**System Response**
3	User clicks "Log Out" but session already expired.	
4		System redirects user to login page and displays message: "Session expired. Please log in again."
Process resumes at Step 2 of **[UC-01: Login]**		
Alternate Course of Action (System Error)		
S#	**Actor Action**	**System Response**
4		System encounters error during logout.
5		System displays error message and prompts user to try again.

Process resumes at Step 3 of the typical flow.

2.8 Manage Candidate Profile

Identifier	UC-08
Name	Manage Candidate Profile
Summary	This use case describes the process of a logged-in Candidate managing and updating their profile in the Swift Hire. Includes [UC-09: Upload CV] and [UC-10: Set Preferences].
Priority	High
Actor(s)	Candidate
Pre-condition(s)	The user must be logged into their Swift Hire account. The system must have the user's account stored and accessible
Post-condition(s)	The user's profile information is updated successfully. If the user chooses to upload a CV or set preferences, those actions are completed through the included use cases and saved in the system.

Typical Course of Action

S#	Actor Action	System Response
1	Candidate navigates to the profile section.	
2		System retrieves and displays the user's current profile information.
3	Candidate selects the option to edit profile.	
4		System displays editable profile fields and profile management options.
5	Candidate updates basic profile information (name, contact details, etc.).	
6		System validates the entered information.

7	Candidate chooses to upload CV.	
8		System invokes UC-09: Upload CV (<<include>>).
9	Candidate chooses to set or update preferences.	
10		System invokes UC-10: Set Preferences (<<include>>).
11	Candidate confirms and saves the profile changes.	
12		System stores updated profile information in the database.
13		System displays confirmation message.
Alternate Course of Action (Incomplete Data Entered)		
S#	**Actor Action**	**System Response**
5	User enters incomplete information.	
6		System displays "Please enter complete information".
7	User re-enters complete information.	
Process resumes at Step 11 of the typical flow.		
Alternate Course of Action (Profile Editing Cancelled)		
S#	**Actor Action**	**System Response**
7	User cancels profile editing.	
8		System discards changes and retains previous profile data.

2.9 Upload CV

Identifier	UC-09
Name	Upload CV
Summary	This use case describes the process of a Candidate uploading their CV in the Swift Hire. Invoked via [UC-08: Manage Profile].

Priority	High
Actor(s)	Candidate
Pre-condition(s)	The candidate must be logged into their Swift Hire account and must be in the Manage Profile section (invoked from UC-08). The CV file must exist on the candidate's device and be in a supported format (e.g., PDF).
Post-condition(s)	The CV file is successfully uploaded and stored in the system. The system parses the CV using the parsing engine and extracts relevant keywords, skills, and information, which are saved in the candidate's profile to enable AI job matching.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Candidate selects the option to upload CV from profile section.	
2		System displays file upload interface.
3	Candidate selects CV file from their device.	
4		System validates file format and size
5	Candidate confirms upload.	
6		System uploads and stores the CV file.
7		System parses the CV and extracts relevant data (skills, experience, etc.).
8		System updates candidate profile with parsed CV data.
9		System displays confirmation message that CV was uploaded successfully.

Alternate Course of Action (Unsupported File Format)		
S#	**Actor Action**	**System Response**
3	Candidate selects unsupported file format.	

4		System displays "Please add a PDF file".
5	Candidate selects valid file format.	
Process resumes at Step 5 of the typical flow.		
Alternate Course of Action (File Size Exceeded)		
S#	**Actor Action**	**System Response**
3	Candidate selects file exceeding allowed size.	
4		System displays "File size exceeds the allowed limit."
5	Candidate selects file size within range.	
Process resumes at Step 5 of the typical flow.		

2.10 Set Preferences

Identifier	UC-10
Name	Set Preferences
Summary	This use case describes the process of a Candidate setting or updating their job preferences in the Swift Hire. Invoked via [UC-08: Manage Profile].
Priority	High
Actor(s)	Candidate
Pre-condition(s)	The candidate must be logged into their Swift Hire account and must be accessing the Manage Profile section (invoked from UC-08). The candidate profile must exist in the system.
Post-condition(s)	The candidate's preferences such as job type, location, and shift are successfully saved in the system. These preferences are used by the system to automatically filter and match relevant job postings.
Typical Course of Action	
S#	**Actor Action**	**System Response**

1	Candidate selects the option to set preferences from profile section.	
2		System displays preference settings form.
3	Candidate enters or selects preferences (e.g., job location, shift type, remote/on-site).	
4		System validates the entered preference data.
5	Candidate confirms and saves preferences.	
6		System stores preferences in the database.
7		System updates candidate profile with new preferences.
8		System displays confirmation message.
Alternate Course of Action (Invalid Data Entered)		
S#	**Actor Action**	**System Response**
3	Candidate enters invalid or unsupported preference values.	
4		System displays "Invalid preference values".
5	Candidate enters values that meet requirements.	
Process resumes at Step 5 of the typical flow.		
Alternate Course of Action (Setup Cancelled)		
S#	**Actor Action**	**System Response**
5	Candidate cancels preference setup.	
6		System discards changes and keeps previous preferences unchanged.

2.11 View Job Postings

Identifier	UC-11
Name	View Job Postings
Summary	This use case describes the process of a Candidate viewing automatically filtered job postings in the Swift Hire.
Priority	High
Actor(s)	Candidate
Pre-condition(s)	The candidate must be logged into their Swift Hire account. The candidate must have a profile with uploaded CV and/or preferences stored in the system. Job postings must exist in the system database.
Post-condition(s)	The system displays a list of relevant job postings filtered and ranked based on the candidate's CV data and preferences. The candidate can view job details and matching information.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Candidate navigates to the job postings section.	
2		System retrieves candidate CV data and preferences.
3		System applies matching algorithm to compare candidate profile with job postings.
4		System filters and ranks relevant job postings.
5		System displays list of matched job postings.
6	Candidate selects a job posting to view details.	
7		System displays detailed job information (role, requirements, location, match score).
Alternate Course of Action (No CV Uploaded/Preferences Set)		

S#	Actor Action	System Response
2		System detects that candidate has not uploaded CV or set preferences.
3		System prompts candidate to 'complete profile' first.
System redirects the user to **[UC-08: Manage Candidate Profile]** to complete their profile.		
Alternate Course of Action (No Matching Jobs Found)		
S#	**Actor Action**	**System Response**
3		System couldn't find a job matching candidate's current profile or preferences.
4		System displays: "No matching jobs found. Try adjusting your preferences."
System redirects the user to **[UC-10: Set Preferences]** to update their preferences.		

2.12 Rate Employer

Identifier	UC-12
Name	Rate Employer
Summary	This use case describes the process of a Candidate providing feedback and rating an employer after completing an interview in Swift Hire.
Priority	Medium
Actor(s)	Candidate
Pre-condition(s)	The candidate must be logged into their Swift Hire account. The candidate must have completed at least one interview scheduled through the system. The system must have stored the interview records for reference.
Post-condition(s)	The employer rating and feedback are successfully saved in the system. The rating can be used for analytics, employer ranking, and future candidate guidance.
Typical Course of Action	

S#	Actor Action	System Response
1	Candidate navigates to the completed interviews section.	
2		System displays a list of interviews attended by the candidate.
3	Candidate selects an interview to rate.	
4		System displays rating and feedback interface.
5	Candidate selects a star rating (e.g., 1–5) and enters optional feedback.	
6	Candidate submits the rating and feedback by clicking 'Submit Rating'.	
7		System validates input and stores the rating and feedback in the database.
8		System confirms submission and updates employer analytics.
Alternate Course of Action (Invalid Feedback)		
S#	**Actor Action**	**System Response**
6	Candidate enters invalid feedback (e.g., too long or unsupported characters).	
7		System displays "Invalid feedback".
8	Candidate enters feedback conforming to the system defined limits.	
Process resumes at Step 6 of the typical flow.		
Alternate Course of Action (Submission Cancelled)		
S#	**Actor Action**	**System Response**
6	Candidate cancels rating submission.	
7		System discards changes and keeps no rating.

Alternate Course of Action (Rating Is Missing)		
S#	**Actor Action**	**System Response**
6	Candidate clicks 'Submit Rating' without selecting a rating.	
7		System displays: "Please select a valid rating (1–5)."
8	Candidate chooses a valid rating.	
Process resumes at Step 6 of the typical flow.		

2.13 Manage Users

Identifier	UC-13
Name	Manage Users
Summary	Allows the Admin to search, view, and manage user accounts, including banning/unblocking users based on low ratings or violations.
Priority	High
Actor(s)	Admin
Pre-condition(s)	The Administrator is logged into the system. The system contains registered users (Candidates, Employers).
Post-condition(s)	The status of one or more user accounts is updated (e.g., banned, restored) or the user list is displayed.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Navigates to the "Manage Users" section.	
2		Displays a list of users with search/filter options.
3	Admin enters filter criteria (e.g., "Rating < 2 stars")	
4	Admin clicks "Search".	
5		Retrieves and displays users matching the criteria.

6	Selects a specific user from the results.	
7		Shows detailed user profile (activity, ratings, history).
8	Admin chooses an action (approve, block, delete).	
9	Confirms the action.	
10		Validates the action.
11		Displays confirmation message "User status updated successfully" and updates the account's status.
Alternate Course of Action (No Matching Users)		
S#	**Actor Action**	**System Response**
5		Displays a message: "No users found matching the criteria."
6	Admin updates the criteria.	
Process resumes at Step 4 of the typical flow.		

Alternate Course of Action (No Users Exist)		
S#	**Actor Action**	**System Response**
2		Displays a message: "No users exist."
Alternate Course of Action (Account Already Approved/Banned/Deleted)		
S#	**Actor Action**	**System Response**
10		System detects the account is already affected by the selected action and displays a message: "Action already active".
11	Admin chooses a valid(inactive) action.	
Process resumes at Step 9 of the typical flow.		

2.14 View System Reports

Identifier	UC-14
Name	View System Reports
Summary	Allows the Admin to view system-generated reports (user activity, hiring analytics, ratings, etc.).
Priority	Medium
Actor(s)	Admin
Pre-condition(s)	Admin is logged into the system. Reports exist in the database.
Post-condition(s)	Reports are displayed for review.

Typical Course of Action

S#	Actor Action	System Response
1	Admin navigates to "System Reports."	
2		Displays available report categories (users, jobs, ratings, analytics).
3	Admin selects a report category.	
4		System prompts for any additional parameters (date range, user type, etc.).
5	Admin sets the desired parameters.	
6	Admin clicks "Generate Report".	
7		Retrieves and displays the relevant report.
8	Admin reviews the report details.	
9		Provides options to export or filter the report.
10	Admin applies the desired filters.	
11	Admin clicks "Filter Report".	
12		Display the filtered reports.
13	Admin selects "Export Report".	

14		Generates and downloads the file.
Alternate Course of Action (No Reports Exist)		
S#	**Actor Action**	**System Response**
2		System displays "No reports exist"
Alternate Course of Action (Invalid Filter Applied)		
S#	**Actor Action**	**System Response**
11	Admin applies invalid filter and clicks "Filter Report".	
12		System displays error message: "Invalid filter. Please enter valid criteria."
13	Admin reapplies filter matching the system requirements.	
Process resumes at Step 11 of the typical flow.		

2.15 Send Interview Reminders

Identifier	UC-15
Name	Send Interview Reminders
Summary	This use case describes the automated process where the System sends interview reminder notifications to Candidates and Employers before a scheduled interview time using cron job logic.
Priority	High
Actor(s)	System
Pre-condition(s)	Interviews must already be scheduled and stored in the database. Valid email addresses must exist for both Candidate and Employer. Cron job scheduler must be operational. SMTP mail server integration must be active.
Post-condition(s)	Reminder email notification is successfully sent to relevant users. Reminder status is logged in the system database.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Cron job triggers at scheduled interval (e.g., every hour).	
2		System fetches interviews scheduled within the next defined reminder window (e.g., 24 hours).
3		System verifies interview status and participant details
4		System generates reminder email content (interview date, time, meeting link).
5		System sends reminder email to Candidate and Employer.
6		System logs reminder delivery status in database.
Alternate Course of Action (Failed Email Attempt)		
S#	**Actor Action**	**System Response**
5		System attempts to send reminder email, but the SMTP service fails.
6		System logs the delivery failure in the database and queues the email for a retry in the next cycle.
Alternate Course of Action (No Interviews Found)		
S#	**Actor Action**	**System Response**
2		System searches the database but finds no interviews scheduled within the reminder window.
3		System logs "No reminders to send" for the current cycle.

2.16 View Hiring Analytics

Identifier	UC-16
Name	View Hiring Analytics
Summary	This use case describes how authorized users (Employer or Candidate) view hiring analytics dashboards within the system.
Priority	Medium
Actor(s)	Candidate, Employer
Pre-condition(s)	User is authenticated. User has appropriate role permissions. Analytics data exists in the system database.
Post-condition(s)	Hiring analytics dashboard is displayed to the user. No system data is modified.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	User clicks on the "Hiring Analytics" tab from their main dashboard.	
2		System verifies role-based access permission.
3		System retrieves relevant analytics data from database
4		System processes and aggregates data (e.g., interview count, job applications, success rate)
5		System displays analytics dashboard with visual charts and statistics.

Alternate Course of Action (No Data Available)		
S#	**Actor Action**	**System Response**
3		System retrieves analytics data but finds no data available.
4		System prepares empty state view.

5		"No Data Available" message displayed.

2.17 Manage Employer Profile

Identifier	UC-17
Name	Manage Employer Profile
Summary	This use case describes the process of a logged-in Employer managing and updating their company profile in Swift Hire.
Priority	High
Actor(s)	Employer
Pre-condition(s)	The user must be logged into their Swift Hire account. The system must have the user's account stored and accessible.
Post-condition(s)	The employer's profile information (e.g., company details) is updated successfully and saved in the system database.

Typical Course of Action

S#	Actor Action	System Response
1	Employer navigates to the profile section.	
2		System retrieves and displays the employer's current profile information.
3	Employer selects the option: "Edit Profile".	
4		System displays editable profile fields (company name, location, description, etc.).
5	Employer updates company profile information.	
6	Employer confirms the changes and clicks "Confirm Changes".	

7		System validates the entered information.
8		System stores updated profile information in the database.
9		System displays confirmation message.
Alternate Course of Action (Invalid Data Entered)		
S#	**Actor Action**	**System Response**
6	Employer enters invalid information and clicks "Confirm Changes".	
7		System displays "Invalid Data Entered".
8	Employer re-enters valid data.	
Process resumes at Step 6 of the typical flow.		
Alternate Course of Action (Profile Editing Cancelled)		
S#	**Actor Action**	**System Response**
6	Employer cancels profile editing.	
7		System discards changes and retains previous profile data.

3. Non-Functional Requirements

3.1 Performance

3.1.1 The system shall process the recruiter's natural language prompt and return parsed results (skills, experience, location) within 3 seconds under normal load.

3.1.2 The CV parsing engine shall extract text from an uploaded PDF and store the resulting keywords within 5 seconds for files up to 5 MB.

3.1.3 All web pages (dashboards, search results) shall load completely within 2 seconds on an average broadband connection.

3.1.4 The auto-matching algorithm that ranks candidates against a job posting shall return results in under 4 seconds for a candidate pool of up to 10,000.

3.2 Scalability

3.2.1 The system shall support at least 10,000 concurrent active candidate sessions (e.g., viewing job postings, updating profiles) without significant degradation.

3.2.2 The database and application layer shall be horizontally scalable to handle peak loads during job fair events or high-traffic periods.

3.2.3 The email notification service shall be able to queue and send reminders to up to 50,000 recipients per hour without blocking other operations.

3.3 Usability

3.3.1 At least 90% of test users (candidates and recruiters) shall successfully complete their core tasks (upload CV, set preferences, enter hiring prompt) on the first attempt.

3.3.2 Navigation menus and UI elements shall remain consistent across all pages, verified through user interface testing.

3.3.3 The recruiter's natural language input field shall provide clear placeholder examples (e.g., "Need Java dev, 2 years exp, Spring Boot") to guide usage.

3.3.4 Error messages shall be descriptive and provide actionable guidance (e.g., "Invalid file format. Please upload a PDF.").

3.4 Data Validation

3.4.1 The system shall validate all uploaded CV files: only PDF format is accepted; file size shall not exceed 10 MB; corrupted or password-protected files shall be rejected with a clear message.

3.4.2 All user input (text fields, forms) shall be sanitized to prevent injection attacks (SQL, XSS). Special characters in names/addresses shall be handled gracefully.

3.4.3 The recruiter's prompt shall be processed with regex patterns that extract only relevant numbers, skills, and locations; unrecognized input shall be ignored but logged for analytics.

3.4.4 The system shall ensure that email addresses entered during sign-up conform to standard format and are verified via confirmation email.

3.5 Search & Filters

3.5.1 The auto-matching engine shall achieve at least 85% precision and recall when ranking candidates against job requirements, based on a test set of 100 annotated profiles.

3.5.2 Candidate search (by recruiter) shall support filtering by skills, years of experience, location, shift preference, and rating. Results shall be displayed within 3 seconds.

3.5.3 Job posting search (by candidate) shall automatically filter jobs based on their parsed CV keywords and preferences, with the option to manually override filters.

3.5.4 The system shall provide a "recommended jobs" section on the candidate dashboard that updates in real-time when the candidate updates their CV or preferences.

3.6 Security & Access Control

3.6.1 The system shall implement JSON Web Token (JWT) based authentication for secure session management.

3.6.2 The system shall enforce Role-Based Access Control (RBAC) to restrict functionality based on user roles (Candidate, Employer, Admin).

3.6.3 The system shall hash all user passwords using secure hashing algorithms (e.g., bcrypt or Argon2) before storing them in the database.

3.6.4 The system shall validate and sanitize all uploaded PDF files to prevent malicious code execution or injection attacks.

3.6.5 JWT tokens shall expire after a configurable time period to reduce unauthorized session risks.

3.6.6 The system shall prevent brute-force login attempts by implementing rate limiting and account lock mechanisms.

3.7 Notifications

3.7.1 The system shall deliver email notifications with a minimum delivery reliability rate of 95%.

3.7.2 The system shall retry failed email deliveries up to three (3) times before marking as failed.

3.7.3 The system shall log all notification events (sent, failed, retried) in the database.

3.7.4 The system shall send interview reminders within ±5 minutes of the scheduled reminder time.

3.7.5 The system shall ensure that duplicate reminder emails are not sent for the same interview event.

3.8 Account Management

3.8.1 The system shall allow users to update profile information securely after authentication.

3.8.2 The system shall enforce strong password requirements (minimum 8 characters including uppercase, lowercase, number, and special character).

3.8.3 The system shall allow users to reset passwords via secure email verification link.

3.8.4 The system shall deactivate accounts after multiple consecutive failed login attempts (e.g., 5 attempts).

3.8.5 The system shall maintain audit logs of account creation, modification, and deletion activities.

3.9 Job & Candidate Management

3.9.1 The system shall allow Employers to create, update, and delete job postings.

3.9.2 The system shall ensure that job postings are visible only to authorized users.

3.9.3 The system shall allow Employers to filter candidates based on predefined criteria (e.g., skills, experience, status).

3.9.4 The system shall maintain data consistency when job postings are edited or deleted.

3.9.5 The system shall archive deleted job postings for audit and recovery purposes.

3.10 Integration

3.10.1 The system shall integrate with SMTP mail server for sending transactional emails.

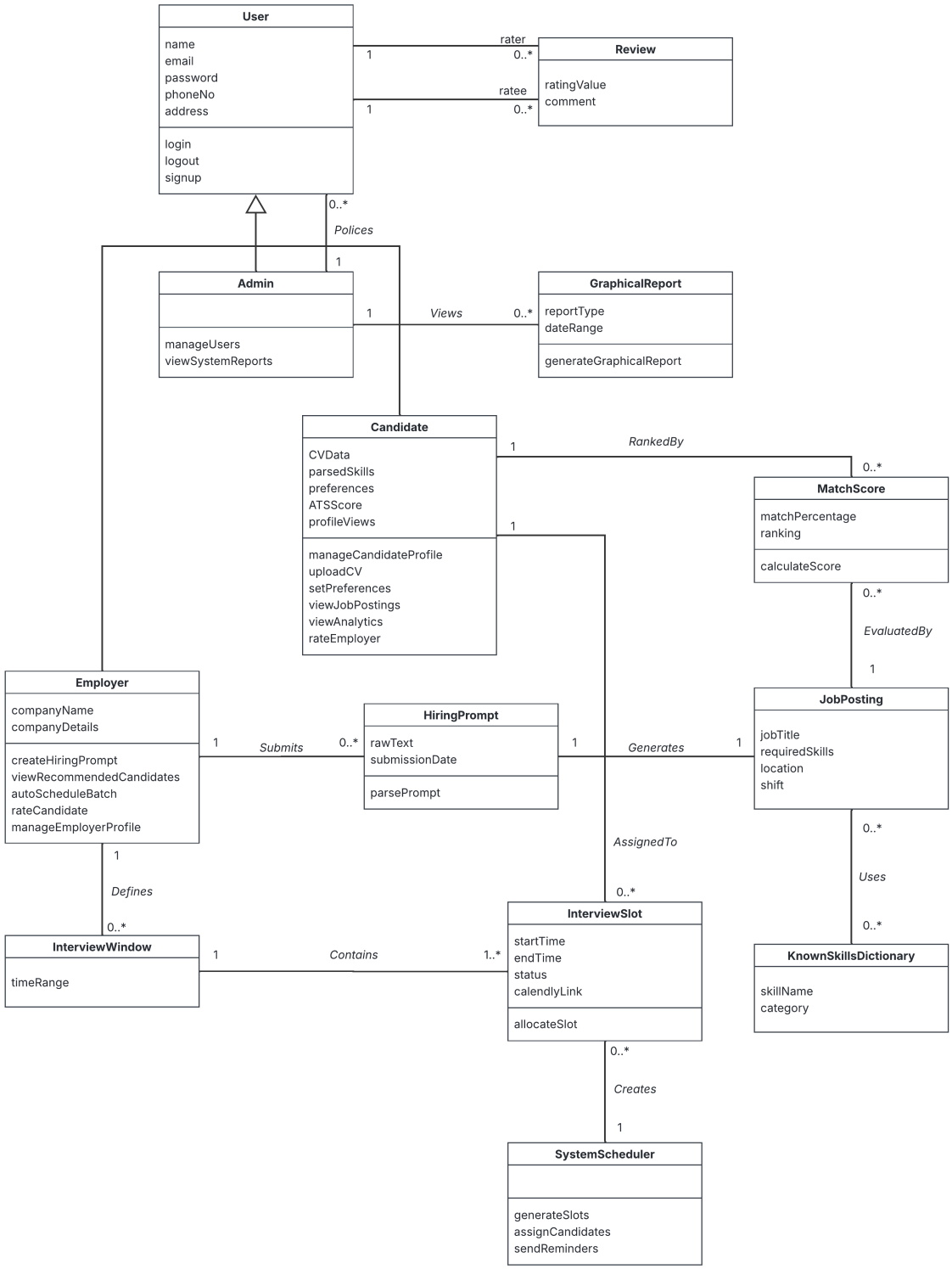
3.10.2 The system shall integrate with third-party scheduling tools (e.g., Calendly) via secure APIs.

3.10.3 All third-party integrations shall use encrypted HTTPS communication.

3.10.4 The system shall handle integration failures gracefully and log error responses.

3.10.5 The system shall validate external API responses before processing data.

4. ACD


User
name
email
password
phoneNo
address
login
logout
signup
rater
1
0..*
Review
ratingValue
comment
ratee
1
0..*
0..*
Polices
1
Admin
manageUsers
viewSystemReports
1
Views
0..*
GraphicalReport
reportType
dateRange
generateGraphicalReport
Candidate
CVData
parsedSkills
preferences
ATSScore
profileViews
manageCandidateProfile
uploadCV
setPreferences
viewJobPostings
viewAnalytics
rateEmployer
1
RankedBy
0..*
MatchScore
matchPercentage
ranking
calculateScore
0..*
EvaluatedBy
1
Employer
companyName
companyDetails
createHiringPrompt
viewRecommendedCandidates
autoScheduleBatch
rateCandidate
manageEmployerProfile
1
Submits
0..*
HiringPrompt
rawText
submissionDate
parsePrompt
1
Generates
1
JobPosting
jobTitle
requiredSkills
location
shift
1
Defines
0..*
AssignedTo
0..*
0..*
Uses
0..*
InterviewWindow
timeRange
1
Contains
1..*
InterviewSlot
startTime
endTime
status
calendlyLink
allocateSlot
KnownSkillsDictionary
skillName
category
0..*
Creates
1
SystemScheduler
generateSlots
assignCandidates
sendReminders